

AI-Ready Customer Support Dashboard

Build a small AI-ready customer support dashboard. The goal is to build a clean, maintainable application that could realistically serve as the foundation for a production product.

At a Glance

Item	Details
Deliverable	AI-ready customer support dashboard (frontend + backend + database)
Core features	Auth, dashboard, conversations (CRUD + history), document metadata upload, conversation search
Clarifications	Up to 2 questions allowed; otherwise document your assumptions
Judged on	Clean, maintainable, well-reasoned design over feature completeness
Submit	Repo, README, schema, API docs, ENGINEERING_REPORT.md, video walkthrough

Tech Stack

Priority	Technology	Layer
Required	Next.js (App Router)	Frontend
Required	FastAPI	Backend
Required	PostgreSQL	Database
Preferred	SQLAlchemy	ORM
Preferred	Tailwind CSS	Styling
Preferred	TypeScript	Language

You may use any additional libraries that you believe improve the solution.

Functional Requirements

Authentication

Implement a simple authentication flow. A basic login system is sufficient. Choose an approach you believe is appropriate.

Dashboard

Display

- Recent conversations
- Uploaded documents
- Basic user information

Conversations

Users should be able to

- Create conversations
- Rename conversations
- Delete conversations
- View conversation history

Persist all conversation data in the database.

Document Upload

Allowed file types

- PDF
- DOCX
- TXT

Only store document metadata. Actual AI document processing is **not required**.

Search

Provide search functionality for conversations. The implementation is your choice.

Backend

Build REST APIs with

- Proper validation
- Appropriate HTTP status codes
- Meaningful error handling

Database

Design an appropriate relational schema. Use whatever structure you believe is most suitable.

UI

The interface should be

- Responsive
- Simple

- Clean
- Easy to use

No advanced animations are expected.

Engineering Decisions

Not every implementation detail has been specified intentionally. You are expected to make reasonable engineering decisions regarding topics such as:

Area	Area
Project structure	Search implementation
Folder organization	State management
Authentication strategy	Error handling
Validation	API response design

Document important assumptions where appropriate.

Clarifications

Software projects often involve ambiguous requirements.

If you believe clarification is necessary, you may ask **up to two clarification questions** during the assessment.

If you choose not to ask questions, document any important assumptions you make.

Submission Checklist

Deliverable	Notes
Source code	GitHub repository
README	Setup instructions
Database schema	Diagram or documentation
API documentation	Endpoints, validation, status codes
ENGINEERING_REPORT.md	Sections outlined below
Video walkthrough	Full app; publicly shared link

ENGINEERING_REPORT.md

Please include the following sections.

1. Project Overview

Provide a short overview of your implementation.

2. Architecture

Explain your overall architecture and project structure.

3. Engineering Decisions

Describe three important technical decisions. For each, explain:

For each decision	Explain
What you chose	The approach or technology you settled on
Alternatives considered	Other options you weighed
Why you selected it	The reasoning behind your final choice

4. AI Collaboration

Describe:

- AI tools used
- Typical workflow
- One example where AI significantly helped
- One example where AI produced an incorrect solution
- How you validated AI-generated code

5. Challenges

Describe the most difficult technical challenge you encountered and how you solved it.

6. Known Limitations

List any known issues, shortcuts, or areas you would improve.

7. Reflection

Briefly answer:

- What did you learn?
- If you had one additional day, what would you improve?
- What part took the longest?

Feel free to modify this structure if another format better explains your work.

Git

Please commit your work regularly using meaningful commit messages.

Important

We do not expect a perfect application.

A clean, maintainable, and well-reasoned implementation is generally stronger than a feature-complete solution with poor design.

During the technical interview, we'll discuss your implementation in depth. You'll be expected to explain your design decisions, AI usage, and engineering choices, so please ensure you understand everything you submit.

If you have any questions, feel free to reach out.

Good luck. We're looking forward to seeing your solution!